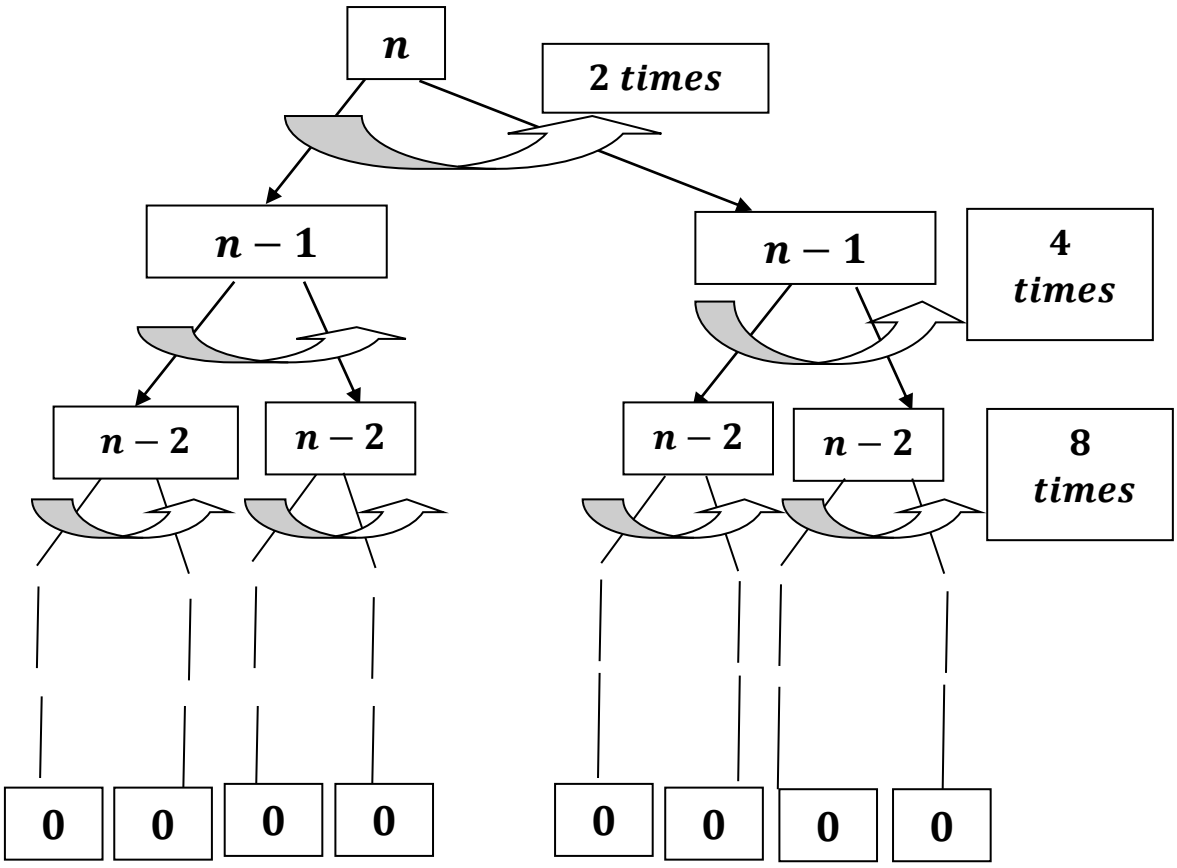
Tower Of Hanoi – Time Complexity calculation through Recursion Tree Method.



***And recurrence relation given as follows*:**

$$T(n) = \begin{cases} 1 & , if\ n = 0 \\ 2T(n-1) + 1 & , if\ n > 0 \end{cases}$$

***Recalling the program*:**

```
void toh(int n, char src, char dest, char aux)
{
    if (n == 0)
    {
        return ;
    }

    toh(n - 1, src, aux, dest);

    cout << "Move " << n << " from " << src << " to " <<
dest << endl;

    toh(n - 1, aux, dest, src);
}
```

Takes 1 unit of time to run.

Takes $T(n-1)$ time to run.

Takes 1 unit of time , when $n > 0$ during recursion.

Takes $T(n-1)$ time to run.

***Here is exactly how the code maps to the math*:**

1. $toh(n-1,\dots)$***(called twice)*: *The function calls itself two times, and in both cases, the problem size shrinks by exactly* 1. *This gives us the* $2T(n-1)$ *part.***

2. $cout \ll$ Move …: ***Between the recursive calls, the CPU does a constant amount of work printing the move. This gives us the*** $+\ 1$ ***part.***

3. $if\ (n == 0) return;$: ***The base case occurs when*** $n = 0$, ***taking a constant amount of work. This gives us*** $T(0) = 1$.

Here $+1$ ***part is the Work per problem size or Cost per problem size.***

$Therefore, the\ Work\ per\ Problem\ is\ Constant\ (C).$

$Level\ (k)$	$No.of\ problems\ (2^k)$	$Problem\ Size\ (N-k)$	$Work\ /Cost\ Per\ Problem\ (Print\ and\ Add)$	$Work\ done = Work\ per\ Problem \times No.of\ Problems$
0	1	n	C	$1 \times C = C$
1	2	$n-1$	C	$2 \times C = 2C$
2	4	$n-2$	C	$4 \times C = 4C$
3	8	$n-3$	C	$8 \times C = 8C$
. .	. .	. .	. .	. .
k	2^k	$n-k$	C	$2^k \times C$
$n\ (Base\ Case)$	2^n	$n-n=0\ (Base\ Case)$	$T(0)$	$2^n \times T(0)$

$As\ Level = k\ and\ Base\ case = 0\ , \therefore n-k=0\ or\ n=k.$

$\therefore when\ level\ reaches\ `n`\ it\ comes\ to\ Base\ Case\ .$

$Now\ , we\ can\ see\ growth : C + 2C + 4C + \cdots + 2^kC + \cdots + 2^nC.$

$\Longrightarrow C(1 + 2 + 4 + \cdots + 2^k + \cdots + 2^n)$

$or, C(1 + 2 + 4 + \cdots + 2^n)$

Applying geometric progression(series) :

$$S(n) = ar^0 + ar^1 + ar^2 + \cdots + ar^n = a\left(\frac{r^{n+1} - 1}{r - 1}\right), where$$

$S(n)$ *is sum of first n terms ,* a *is coefficient and* r *is common ratio between the consecutive terms.*

We get:

$$1 \times 2^0 + 1 \times 2^1 + 1 \times 2^2 + \cdots + 1 \times 2^n$$

$$\Rightarrow 1\left(\frac{2^{n+1} - 1}{2 - 1}\right)$$

$$\Rightarrow 2^{n+1} - 1$$

$$\therefore Total\ Work = 2^{n+1} - 1$$

putting Big O we get:

$$\Rightarrow O\ (2^{n+1} - 1)$$

$$\Rightarrow O(2^{n+1}) - O(1)$$

$$\Rightarrow O(2^{n+1})$$

$\Rightarrow \boldsymbol{O(2^n \times 2)}$

$\Rightarrow \boldsymbol{O(2^n)} \times \boldsymbol{O(2)}$

$\Rightarrow \boldsymbol{O(2^n \times 2)}$

$\Rightarrow \boldsymbol{O(2^n)}.$

****